DeepTrustxAI

Module 3

Overview of Honeypots

Deception Engineering & Threat Capture

Instructor: Abhishek | Duration: 2 Hours

Let's Start With a Question

DeepTrustxAI

You have locked every door in your house.

- Alarm on every window? **Yes.**
- Camera at the front gate? **Yes.**
- But what if you left one door **unlocked on purpose** — and hid a camera behind it?

The moment a thief walks through that door, you **know exactly who they are, what tools they carry, and what they want.**

That unlocked door is a Honeypot.

What is a Honeypot?

DeepTrustxAI

A **honeypot** is a fake system placed on your network to **attract and trap attackers.**

- Looks exactly like a real server — SSH, FTP, HTTP, SMB
- Has **no legitimate users** — any connection is suspicious by definition
- Lets you **safely observe attacker behavior** without risk to real systems
- Collects real attacker tools, credentials, and techniques

Golden rule: a honeypot that nobody knows about except you is a perfect trap.

Why Use Honeypots in a SOC?

DeepTrustxAI

- **Zero false positives** — no legitimate user should ever touch a honeypot
- Detect **lateral movement** inside your own network — attacker hits the trap
- Collect real **Indicators of Compromise** (IPs, tools, credentials, payloads)
- Understand **who is targeting you** and what they are after
- Every honeypot alert = **highest confidence alert** in your SIEM

While SIEM rules generate noise, **honeypot alerts are always real.**

Low vs High Interaction Honeypots

DeepTrustxAI

Feature	Low Interaction	High Interaction
What it is	Simulates a service	Full real OS/application
Risk level	Very low	Higher — real exploit possible
Data captured	Connection attempts	Full attacker session
Examples	Cowrie, Dionaea	Full VM with real SSH
Best for	Large-scale detection	Deep research

We use **low-interaction** honeypots in this course — safe, scalable, and easy to deploy.

Our Honeypot Platform: T-Pot

DeepTrustxAI

T-Pot is an all-in-one honeypot platform running **20+ services** in Docker containers.

- Deploys on a single machine — even a **Raspberry Pi**
- Each honeypot service listens on **different ports and protocols**
- All logs collected centrally under `/data/tpot/`
- Integrates with **Wazuh** via agent for centralized alerting

T-Pot data root: `/data/tpot/`

Compose file: `docker-compose.yml`

Check services: `sudo docker ps`

T-Pot Services: What Each Does

DeepTrustxAI

Service	Simulates	Port(s)
Cowrie	SSH / Telnet server	22, 23
Dionaea	SMB, FTP, HTTP, MySQL	21, 445, 3306
H0neytr4p	HTTPS web server	443
Conpot	ICS / SCADA devices	102, 161, 502
Heralding	Email, RDP, VNC	110, 3389, 5900
Suricata	IDS on all honeypot traffic	All interfaces

Each service writes JSON logs to `/data/tpot/<service>/log/`

Cowrie: The SSH Honeypot

DeepTrustxAI

Cowrie emulates a real SSH and Telnet server.

When an attacker connects:

- Every **username and password** they try is logged
- If they “log in”, they get a **fake shell** — all commands recorded
- Any **files they download or upload** are captured for analysis
- Full **session replay** available — watch exactly what they did

Log path: `/data/tpot/cowrie/log/cowrie.json`

Key fields: `eventid, src_ip, username, password, input`

Brute force on port 22 is the most common thing Cowrie sees — within minutes of going live.

Dionaea: The Multi-Protocol Honeypot

DeepTrustxAI

Dionaea emulates vulnerable Windows services.

- Port **445 (SMB)** — catches EternalBlue / WannaCry-style exploits
- Port **3306 (MySQL)** — catches credential stuffing attempts
- Port **21 (FTP)** — catches brute force and file upload attempts
- **Captures malware binaries** dropped by attackers

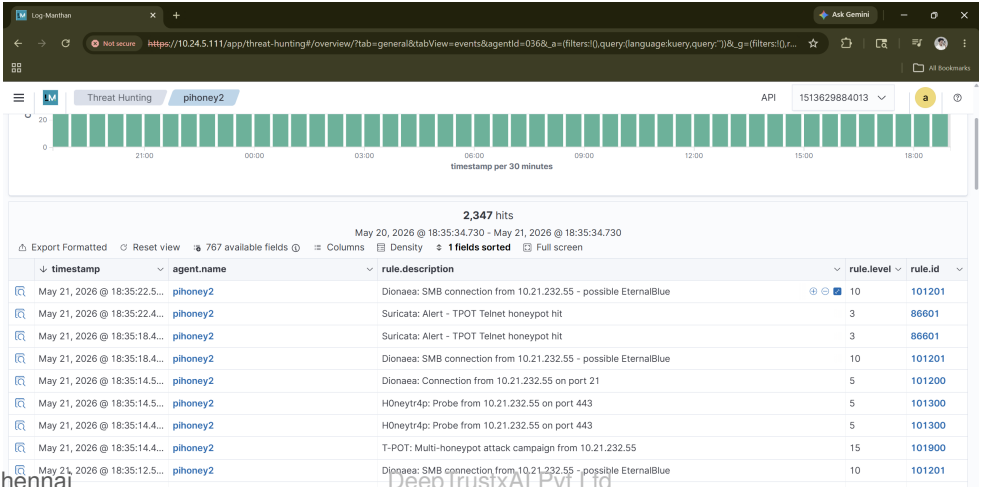
Log path: `/data/tpot/dionaea/log/dionaea.json`

Key fields: `connection, transport, src_ip, dst_port`

EternalBlue on port 445 = one of the most common honeypot alerts you will see in the wild.

Live Honeypot Attacks in Wazuh

DeepTrustxAI



Reading a Honeypot Alert

DeepTrustxAI

From the live dashboard (previous slide), one row tells us everything:

rule.description	Dionaea: SMB connection — possible EternalBlue
rule.level	10 (High)
rule.id	101201
agent.name	pihoney2 (the honeypot machine)
src_ip	10.21.232.55 (the attacker)

Key pattern: same IP **10.21.232.55** appears across Dionaea **and** H0neytr4p **and** Suricata.

One IP hitting multiple services = coordinated scan. Rule 101900 fires: “T-POT multi-honeypot attack campaign”. Level 15 — Critical.

What to Look for in Honeypot Alerts

DeepTrustxAI

- **Cowrie login.failed** — credential lists being tried on SSH port 22
- **Dionaea EternalBlue (101201)** — automated SMB exploit scanning
- **T-POT campaign (101900)** — one IP hitting **multiple** services = serious actor
- **H0neytr4p probe on 443** — HTTPS scanning for web vulnerabilities
- **Same source IP across services** — pivot from scan to targeted attack

Rule 101900 is the most important — it fires only when one IP triggers multiple honeypot services within a short window.

Where to Place Honeypots

DeepTrustxAI

Placement determines what you catch:

- **Internet-facing** — catches external scanners, botnets, opportunistic attackers
- **Inside the network** (DMZ or internal VLAN) — detects **lateral movement**
- **Near high-value assets** — database subnet, AD domain controllers

Critical rule: Never put a honeypot **on the same subnet** as production servers without firewall isolation.

A compromised honeypot must not be able to pivot to real systems.

Simulating Cyber Attacks on Honeypots

DeepTrustxAI

To understand what attackers do, we **simulate the same attacks ourselves.**

Network-based attacks we will demonstrate:

- **Nmap scan** — discover open ports on the honeypot
- **SSH brute force** — credential stuffing against Cowrie (port 22)
- **SYN flood DoS** — flood packets, watch Suricata fire
- **SMB probe** — EternalBlue-style scan against Dionaea (port 445)
- **HTTP directory scan** — Gobuster/Nikto against H0neytr4p (port 443)

For each attack: run it → see the alert in Wazuh → understand what fired and why.

Attack 1: Nmap Port Scan

DeepTrustxAI

What it does: Discovers which ports are open on the target.

Run against the honeypot:

```
nmap -sS -p 21,22,23,443,445,3306 <honeypot-ip>
```

What Wazuh sees:

- Suricata fires: *“ET SCAN Nmap Scripting Engine”*
- Dionaea logs connection attempts on each scanned port
- Cowrie logs the TCP knock on port 22

Suricata log: `/data/tpot/suricata/log/eve.json`

Attack 2: SSH Brute Force on Cowrie

DeepTrustxAI

What it does: Tries hundreds of username/password combinations against SSH.

```
hydra -l root -P /usr/share/wordlists/rockyou.txt
ssh://<honeypot-ip>
```

What Wazuh sees:

- Cowrie logs every attempt: `cowrie.login.failed`
- Rule 101101 fires for each failure (level 7)
- After threshold: brute-force correlation rule fires (level 10)

Cowrie log: `/data/tpot/cowrie/log/cowrie.json`

Attack 3: H0neytr4p Web Probe

DeepTrustxAI

What it does: Scans HTTPS port 443 for exposed sensitive files.

```
# Basic probe (rule 101300, level 5)
curl -sk https://<honeypot-ip>/admin -o /dev/null

# Trapped paths (rule 101301, level 12)
curl -sk https://<honeypot-ip>/.env -o /dev/null
curl -sk https://<honeypot-ip>/wp-config.php -o /dev/null
```

What Wazuh sees:

- Rule 101300 — `H0neytr4p: Probe` (level 5) for normal paths
- Rule 101301 — `H0neytr4p: TRAPPED attacker` (level 12) for sensitive paths
- `trapped_for` field tells you exactly which pattern matched

H0neytr4p log: `/home/admin/tpotce/data/h0neytr4p/log/log.json`

Analyzing Attack Patterns in Logs

DeepTrustxAI

After running the attacks, we **analyze the captured data** in Wazuh.

Key questions to answer from the logs:

- Which **usernames and passwords** did the attacker try? (Cowrie)
- Which **ports** were scanned? (Dionaea + Suricata)
- Did the attacker try **multiple services**? (cross-service correlation)
- What **tools** did they use? (User-agent, scan timing, packet patterns)
- What is the **source IP**? Is it on a threat intel blocklist?

WQL query to find all honeypot events in the last hour:

```
rule.groups:honeypot AND rule.level>=7
```

Key Takeaways — Module 3

DeepTrustxAI

- Honeypots are **traps** — any connection to them is always an alert
- **T-Pot** bundles 20+ honeypot services in one Docker deployment
- **Cowrie** catches SSH brute force; **Dionaea** catches SMB/EternalBlue
- All logs flow into **Wazuh** via `<localfile>` entries in `ossec.conf`
- Simulate attacks yourself to **understand what you will face**
- Honeypot alerts = **zero false positives** — treat every one seriously

Security can only be understood when you make it very simple.
If you can explain a honeypot to a 10-year-old, you truly understand it.

Coming Next: Practicals Session

DeepTrustxAI

In the Practicals session you will:

- **Deploy** T-Pot on your lab VM using Docker Compose
- **Connect** T-Pot logs to the Wazuh agent
- **Run** Nmap, SSH brute force, and SMB probe attacks
- **Watch** the alerts appear live in the Wazuh Dashboard
- **Analyze** the captured credentials, IPs, and attack patterns
- **Identify** if the same attacker hit multiple services

Bring your lab sheet. Every attack you run should produce a visible alert.